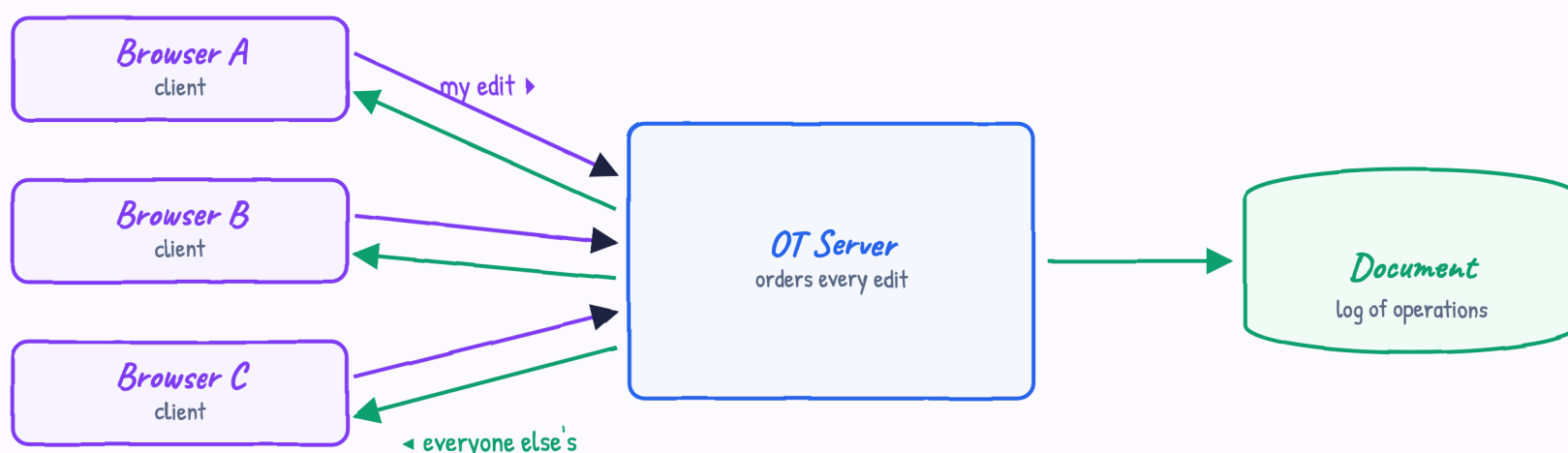




# System Design of Google Docs

How millions edit the same document at once

Operational Transformation, explained properly



## What's inside

- ✓ What are we building?
- ✓ The core problem
- ✓ Operational Transformation
- ✓ OT vs CRDT
- ✓ Architecture
- ✓ History, offline and presence
- ✓ Interview questions

## Who it's for

- ✓ Backend engineers preparing for system design
- ✓ Anyone asked "design a collaborative editor"
- ✓ Developers curious how Docs actually works

## Why it matters

- ✓ The real algorithm, not a hand-wave
- ✓ A diagram for the part everyone gets wrong
- ✓ Interview answers you can say out loud

Comment DOCS and I will DM you the full PDF 



# System Design of Google Docs

How millions edit the same document at once

Operational Transformation, explained properly

1

What are we building?

Design a document everyone can edit at the same time



## The product

A document that many people edit at once, in the browser, and everyone sees every keystroke within a moment.

- Type and see others typing live
- Cursors and selections of other people are visible
- Works after you close the laptop and reopen it
- Full history — go back to any earlier version



## Must-haves

- Convergence — everyone ends up with the exact same document
- Low latency — a keystroke shows for others in well under a second
- Never lose an edit, even if the network drops mid-word
- Works offline, syncs when the connection returns

1

## Out of scope



- Printing and export formats
- Comments and suggestions mode
- Permissions and sharing UI

2

## The hard part



- Two people editing the same sentence in the same second
- Neither has seen the other's change yet
- Both must still end up identical

3

## Why it is hard



- Every edit is relative to a position
- Positions shift when someone else types
- So an old edit means the wrong thing

4

## What this is not



- Not last-write-wins — that loses work
- Not file locking — that kills collaboration
- Not merging on save — too late



# System Design of Google Docs

How millions edit the same document at once

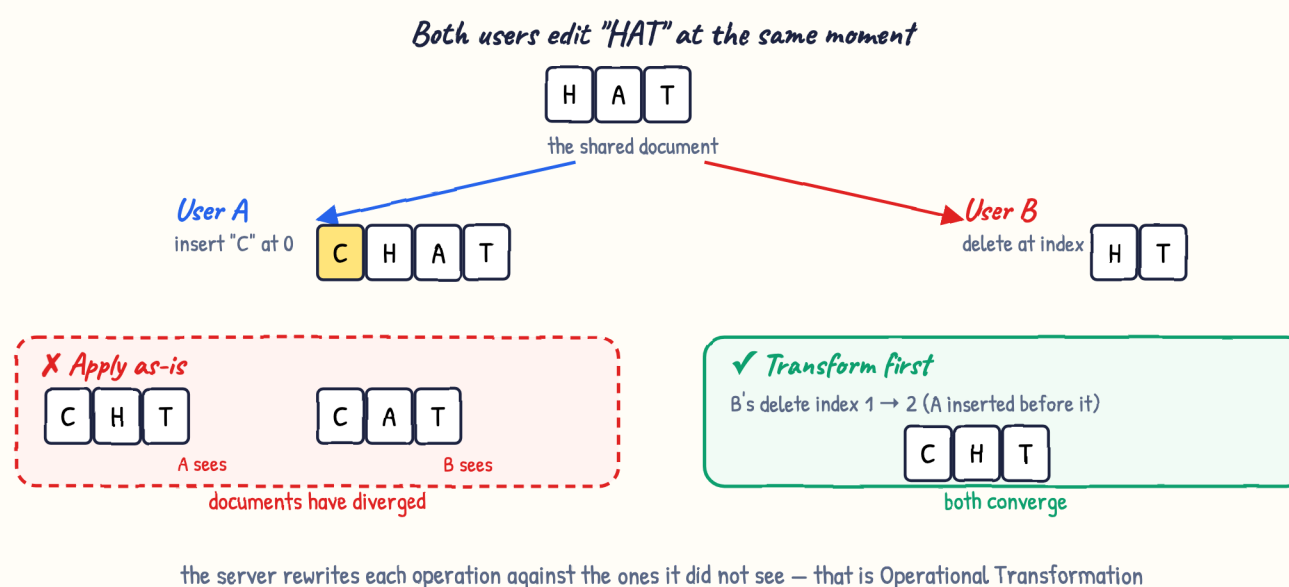
Operational Transformation, explained properly

2

## The core problem

Why two edits cannot just be applied

Concurrent edits on the same word



1

## Why position breaks



- Every edit is "at index N"
- Someone typing before N shifts everything
- Your edit now points at the wrong character

2

## Last-write-wins?



- One person's typing silently disappears
- Unacceptable in a shared document
- Both edits must survive

3

## Locking?



- Lock the paragraph while someone types
- Kills the whole point of collaboration
- Nobody would use it

4

## Merge on save?



- Like git conflict markers, mid-sentence
- Too late — people diverged minutes ago
- Real-time means resolving per keystroke

Key

- ✓ Every edit says "at position N" — and N moves when someone else edits before it

Takeaways

- ✓ Applying both edits as-is gives two different documents
- ✓ Someone has to rewrite the later edit so it still means the same thing



# System Design of Google Docs

How millions edit the same document at once

Operational Transformation, explained properly

3

## Operational Transformation

The idea Google Docs actually runs on



### The rule

Before applying an incoming operation, **rewrite it** against every operation it has not seen yet.

- An edit is an **operation**: insert(pos, char) or delete(pos)
- The server keeps them in **one global order**
- Each client's operation is transformed against that order
- Result: every client applies a different-looking op and lands on the **same text**



### Why a central server

- Someone must decide **which edit came first**
- The server assigns a sequence number to every operation
- It already sees every edit anyway — for access control and storage
- Adds only a few milliseconds, and keeps the document **small**

1

### Client side



- Apply your own edit **immediately** — no waiting
- Send it to the server, keep it 'pending'
- When the ack comes back, reconcile

2

### Server side



- Order the operation
- Transform it against anything newer
- Broadcast to every other client

3

### Why it feels instant



- Your own typing never waits for the network
- **Optimistic** local apply, corrected later
- Correction is usually invisible

4

### The catch



- The transform function must be **provably correct**
- Early OT algorithms had convergence bugs
- This is the hardest code in the system



# System Design of Google Docs

How millions edit the same document at once

Operational Transformation, explained properly

4

## OT vs CRDT

The other way to solve the same problem



Two answers to one question

|                        | Operational Transformation | CRDT                               |
|------------------------|----------------------------|------------------------------------|
| Identifies position by | an integer index           | a globally unique id per character |
| Needs a central server | yes — for ordering         | no                                 |
| Document size          | compact                    | metadata on every character        |
| Deleted text           | gone                       | lingers as invisible tombstones    |
| Best for               | a server-backed editor     | offline-first and peer-to-peer     |
| Used by                | Google Docs                | Figma, many local-first apps       |

1

### Why Google chose OT



- The server has to see every edit anyway
- Keeps the document small on the wire
- Ordering is free once you have a server

2

### Why CRDT is popular now



- Two replicas always converge, in any order
- No server needed at all
- Perfect for offline-first products

3

### The trade



- OT buys compactness, pays with a tricky transform
- CRDT buys simplicity, pays with memory

4

### Interview line



- "OT centralises ordering; a CRDT makes ordering unnecessary. Google Docs already needed the server, so OT was the cheaper trade."



# System Design of Google Docs

How millions edit the same document at once

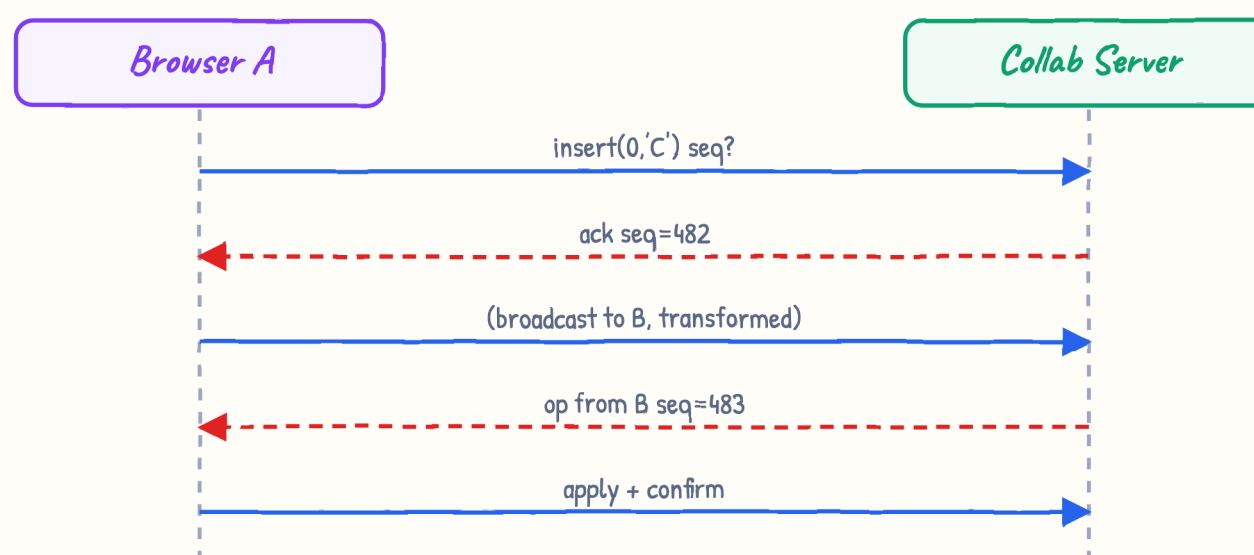
Operational Transformation, explained properly

5

## Architecture

What actually sits between two keyboards

Edit path, end to end



a persistent **WebSocket** in both directions – not request/response

1

### Connection



- A long-lived **WebSocket** per open document
- Server can push the moment an op is ordered
- Falls back to long-polling on bad networks

2

### Collab server



- Owns one document's ordering
- All editors of a doc route to the **same** instance
- Keeps the in-memory state and the pending ops

3

### Revision log



- Every operation appended, never edited
- This is the **source of truth**
- Rebuild the whole doc by replaying it

4

### Snapshots



- Full document saved every few thousand ops
- Open = load snapshot + replay the **tail**
- Otherwise opening an old doc would take forever



# System Design of Google Docs

How millions edit the same document at once

Operational Transformation, explained properly

6

## History, offline and presence

The features that fall out of the log



### Why an append-only log

Because every edit is stored as an operation, three features come almost free.

- Version history — replay up to any point in time
- Unlimited undo — invert the last operation
- Reconnect — send the client only the ops it missed



### Offline editing

- Local edits queue up while disconnected
- On reconnect they are sent and transformed against everything that happened
- Long offline edits can still conflict badly — that is why apps warn you

1

### Presence



- Cursor position is just another tiny message
- Sent on a separate, lossy channel
- Losing one does not matter — the next overwrites it

2

### Never persisted



- Cursors and selections are ephemeral
- No point writing them to the log
- Keeps the log clean and replayable

3

### Awareness cost



- 25 people typing = a lot of tiny messages
- Throttle cursor updates to ~10/sec
- Batch them per tick

4

### Conflict UX



- Most conflicts are invisible — that is the goal
- Users only notice when text jumps
- Good OT makes the jump rare



# System Design of Google Docs

*How millions edit the same document at once*

Operational Transformation, explained properly

7

## Interview Questions

### The answers that land

1

#### How do two edits not conflict?



- "Each edit is an operation with a position. The server orders them and **transforms** each one against the ops it hadn't seen, so everyone converges on the same text."

2

#### Why not last-write-wins?



- "It silently throws away one person's typing. Collaborative editing needs both edits to survive, not one to win."

3

#### OT or CRDT?



- "OT if you already have a server ordering everything — it keeps documents compact. CRDT if you need offline-first or peer-to-peer with no server."

4

#### How is history stored?



- "An append-only operation log plus periodic snapshots. Opening a doc loads the last snapshot and replays the tail."

5

#### Why WebSockets?



- "The server needs to **push** the instant an op is ordered. Polling would add latency on every keystroke."

6

#### What about offline?



- "Queue operations locally, then transform them against everything that happened while you were away when you reconnect."



# System Design of Google Docs

*How millions edit the same document at once*

Operational Transformation, explained properly

*That's Google Docs, end to end.*

What you just went through

- |                                 |                                 |
|---------------------------------|---------------------------------|
| 1 What are we <i>building</i> ? | 5 Architecture                  |
| 2 The <i>core problem</i>       | 6 History, offline and presence |
| 3 Operational Transformation    | 7 Interview Questions           |
| 4 OT vs CRDT                    |                                 |



Save this for your system design round



Share it with someone prepping right now



Comment **DOCS** for the complete PDF in your DM

Written from 10 years of backend engineering — distributed systems,  
real-time, fintech.

@vikstack